Blochchain and Applications

Lab 01: Node Architecture, Clients, and State Synchronization

ndhy@fit.hcmus.edu.vn

Department of Knowledge Engineering

November 6, 2025

Purpose and Outcomes

Purpose

- Explain how a blockchain node is organized and why it is structured that way.
- Connect consensus theory to running software: storage, sync, recovery, serving.

Outcomes

- Trace: message $\rightarrow$ validation $\rightarrow$ execution $\rightarrow$ commit $\rightarrow$ API.
- Evaluate trade-offs of node roles and synchronization modes.

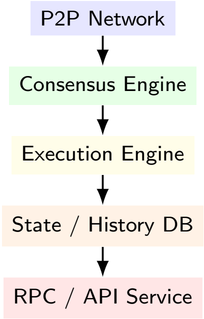
Node Roles

- **Full node:** verifies all blocks, maintains latest state.
- **Archival node:** keeps full history; deep queries and audits.
- **Pruned node:** discards old history; retains recent state.
- **Light client:** headers + proofs on demand.
- **Stateless client:** no state; witnesses for validation.

Node Roles: Comparison

Type	Storage	Verification	Typical Use
Full	Latest state	Self-verified	Validator, operator
Archival	All history	Self-verified	Explorer, analytics
Pruned	Recent state	Self-verified	Lightweight services
Light	Headers	Proof-based	Wallets, mobiles
Stateless	None	Witness-based	Scalability research

Internal Architecture Overview

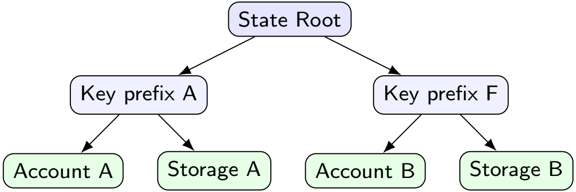


Each layer isolates concerns: networking, agreement, execution, persistence, serving.

Processing Pipeline

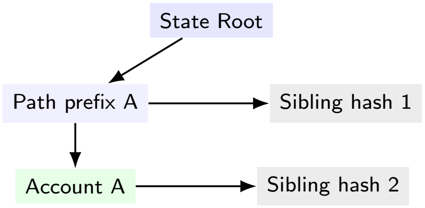
1. Receive header/block from peers (framing, deduplication).
2. Validate header: parent link, timestamp policy, consensus proof.
3. Execute transactions with the state transition function.
4. Update state root and append to history.
5. Serve the new head and receipts to clients.

State Representation: Merkle Patricia Trie



Nodes are hashed; path compression shortens branches. A Merkle proof = path + sibling hashes to recompute the root.

Merkle Proof: What is Sent?



Verifier recomputes hashes bottom-up to check equality with the advertised root.

Block Structure and Receipts

Header: parent hash, state root, receipts root, time, proposer

↓

Transactions (ordered)

↓

Receipts (gas used, logs)

Execution produces receipts and the new state root. Applications read receipts, not raw diffs.

Synchronization Modes

- **Full sync:** replay all blocks; strongest self-verification.
- **Fast / Snap sync:** headers + recent snapshot; skip deep replay.
- **Checkpoint sync:** trust an anchor block hash and verify forward.

Faster startup implies more reliance on trusted anchors or snapshots.

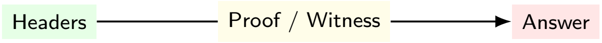
Light vs Stateless Validation

Light client

- Keeps headers; verifies consensus proofs.
- Requests Merkle proofs for queries.

Stateless client

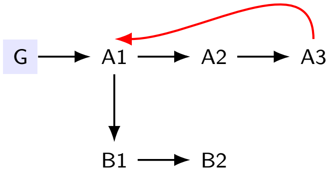
- Keeps no state; witness per TX.
- Proof size and bandwidth dominate cost.



Sync Modes vs Trust Levels

Mode	Historical Work	State Trust	Startup Time
Full replay	Verify all	Self-verified	Slow
Fast / Snap	Headers + snapshot	Trusted recent state	Medium
Checkpoint	From known hash	Trusted anchor	Fast

Handling Reorganizations

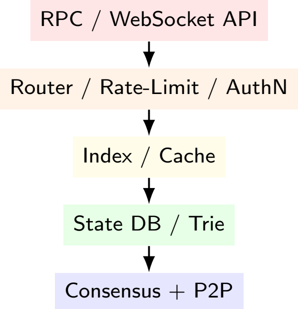


Roll back to common ancestor; re-apply canonical path.

Crash Recovery

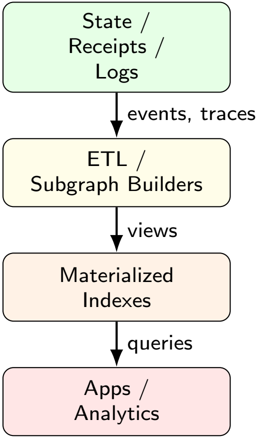
- Write-ahead logs persist intent before state commit.
- On restart, replay WAL and re-validate unfinalized blocks.
- Recovery mirrors databases but respects chain rules.

RPC Stack for Applications



Public vs private endpoints; separate read and sensitive write channels.

Indexing and Caching Pipeline



Indexing converts append-only logs into query-friendly tables.

Security Exposure at the API Edge

- Public RPCs can be abused: flooding, replay, expensive queries.
- Mitigations: quotas, authentication, caching, query shaping.
- Principle: minimize stateful side-effects at public endpoints.

Case Study: Ethereum Clients

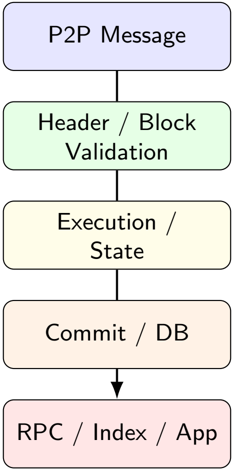
baseline	IO-optimized	DB-focused
Geth (monolithic)	Erigon (modular/ETL)	Nethermind (DB-tuned)

Different philosophies reflect different bottlenecks: CPU, I/O, database layout.

Design Trade-offs

	Option A	Option B
State retention	Archival	Pruned
Sync mode	Full replay	Snap / Checkpoint
Verification	Self-verify	Trusted anchors
Serving	Peer-to-peer	API gateway
Role	Validator	Observer

End-to-End Dataflow



Each box is measurable: latency and error budgets belong here.

Takeaways

- Node architecture turns protocol rules into running systems.
- Sync modes trade verification work for startup speed.
- Clear separation of concerns improves reliability and security.